

Causeway: Restoring Task Accessibility for Instruction Switching in VLA Policies

Qingzi Wang¹, Kaixi Feng¹, Guangyao Shi², Xiyang Wu¹, Ang Li¹, Dinesh Manocha¹

Abstract—Vision-language-action (VLA) policies can execute many tasks from standard initial states, yet a new instruction may fail after another task has altered the robot’s physical state. We study instruction switching, where a new task is issued during or after the execution of a different one. We observe that a target task that is reliably completed from its standard initial states can become inaccessible from states produced by a preceding task. We call such states *task islands*. We propose *Causeway*, a training-free inference-time intervention. Given the current state and a re-entry pose for the target task, Causeway back-propagates through the frozen decoding computation and applies a state-directed write within the action-stream representation. The VLA decodes the return motion itself, without parameter updates, a new action head, or external action generation. Across 71 cross-object pairs, three switch timings, and three VLA architectures on LIBERO-Goal [1], Causeway raises bare-switch success from 3–26% to 47–65% and increases the rate of reaching the handoff neighborhood by 42–72 percentage points across models. Additional experiments on LIBERO-Object and a real xArm platform show that the recovery extends beyond the main LIBERO-Goal setting, both in simulation and on a robot.

I. INTRODUCTION

Vision–Language–Action (VLA) policies leverage pre-trained multimodal representations to execute diverse manipulation tasks specified via natural language [2]–[5]. However, standard benchmarks evaluate tasks from fixed initial distributions [1]. In sequential deployment, completing one task alters the robot and scene state, often rendering subsequent tasks inaccessible. Because resetting between instructions discards progress, practical competence requires that tasks remain accessible from the states that preceding tasks leave behind.

We study *instruction switching*, in which a policy begins task A and permanently receives instruction ℓ_B at an intermediate or terminal state. Although the policy may complete B reliably from its nominal initial states, it can fail to initiate B from states produced by A . We call these states *task islands*. Their occurrence depends on the target task and physical state, not simply on elapsed execution time.

Task islands are closely related to the state-dependent steerability studied by ReSteer [6]. ReSteer measures post-switch success across intermediate states, attributes limited steerability coverage to insufficient task contrast in the training data, and improves the policy through stage-matched transition generation and self-refining behavioral cloning. We address a complementary problem: online recovery after a bare instruction switch has failed. Rather than generating transition data

or updating the policy, we study and recover from inaccessible states in frozen VLAs.

Main Results: We find that task accessibility can decline near contact and grasp events (Sec. III-C). At these states, a new instruction continues to alter the text representation, but its effect on the decoded action weakens substantially (Sec. III-E). The action-stream representation nevertheless remains locally steerable by a state-directed gradient.

We introduce Causeway, a training-free inference-time intervention. Our formulation addresses the failure by guiding the robot toward a demonstration-supported re-entry region for the target task. Given the current state and a re-entry pose derived from target-task demonstrations, we define a target-directed action objective, differentiate it through the frozen decoder, and add the resulting direction to the action-stream representation. Our frozen policy generates the recovery actions, while proprioceptive thresholds govern intervention and handoff. Causeway neither updates model parameters nor replaces policy control with a new action head or external action generation. Moreover, it does not teach the VLA a new task; rather it restores access to an existing capability. Our contributions are threefold:

- 1) We characterize task islands through target-relative state compatibility and connect post-switch failure to demonstration-supported geometry (Sec. III-B), behavioral escape maps (Sec. III-C), and attenuation of instruction influence (Sec. III-E).
- 2) We introduce Causeway, a training-free representation intervention with component-wise pose steering (Sec. IV-A) and state-dependent engagement and hand-off (Sec. IV-B). Our method applies to parallel, flow-matching, and diffusion-based VLA decoders.
- 3) Across 71 cross-object task pairs, three switch timings, and three VLA architectures on LIBERO-Goal ($\pi_{0.5}$, OpenVLA-OFT, and GR00T N1.5), Causeway raises success from 3–26% to 47–65% (Sec. V). Experiments on LIBERO-Object and a real xArm platform (Sec. VI) further test its generality.

II. RELATED WORK

a) VLA policies and instruction switching: VLA policies use language to select behaviors from broad manipulation repertoires [2]–[5], [8]–[10], while standard benchmarks generally evaluate one instruction from a prescribed initial-state distribution [1]. High-level systems compose skills using learned affordances [11], and interactive approaches use language corrections to redirect ongoing behavior [12].

¹University of Maryland, College Park, MD, USA.

²University of Southern California, Los Angeles, CA, USA.

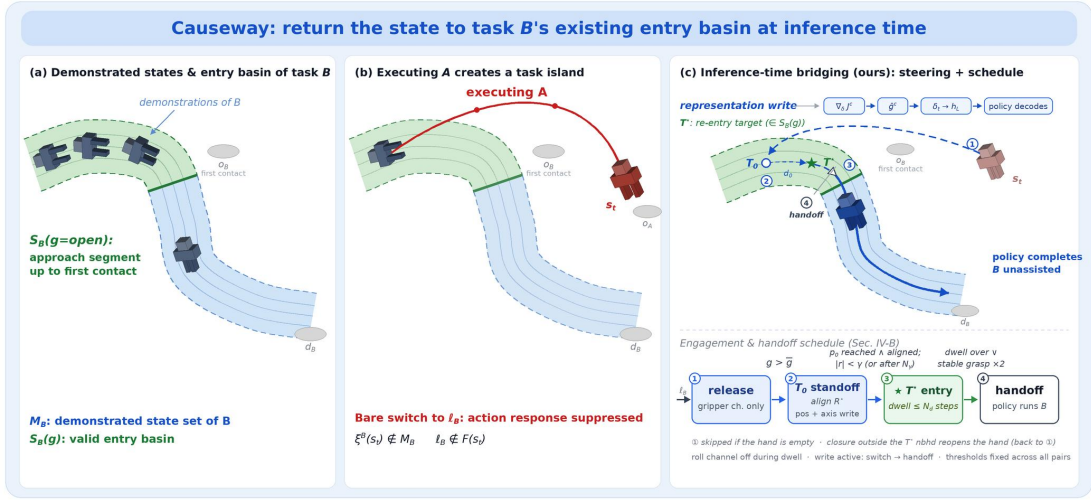


Fig. 1: **Causeway bridges task islands.** (a) Demonstrations of task B define its demonstrated state set M_B , with $S_B(g)$ denoting the valid re-entry region. (b) Executing task A can leave the robot in a state from which task B is no longer accessible, so a bare switch to ℓ_B fails. (c) Causeway steers the action-stream representation with a state-directed gradient (top) until the robot re-enters the basin, scheduled through four phases ①–④ (bottom, Sec. IV-B). Dashed: write active; solid: the frozen policy completes B on its own.

Recent works directly study task switching in VLA policies. SwitchVLA trains an execution-aware policy to select forward, rollback, and advance behaviors using phase and contact supervision [7]. ReSteer formalizes state-dependent steerability, evaluates post-switch success across intermediate states, and improves switching through generated transition data and policy refinement [6]. In contrast, Causeway shares the same observation that instruction responsiveness depends on state, but addresses a different recovery setting. Rather than training switching behaviors or fine-tuning the policy on generated transitions, Causeway uses target-relative geometry to restore access to a target task at inference time. This is consistent with the initiation-set perspective in hierarchical reinforcement learning [13], [14].

b) Inference-time steering and representation intervention: Inference-time guidance modifies model predictions without updating parameters. Classifier-free and contrastive guidance amplify differences between conditional predictions [15], [16]. In robotics, DeLock applies contrastive prompt guidance to the denoising dynamics of a post-trained VLA, using the difference between new and previously trained prompts to preserve responsiveness under novel instructions [17]. Activation-steering methods instead add directions derived from contrasting examples or representations to intermediate model states [18]–[20]. These approaches strengthen an instruction-conditioned signal but do not explicitly determine which physical state would make the target task executable. In contrast, Causeway derives its intervention from the error between the current end-effector pose and a demonstration-supported re-entry pose. Our objective is state recovery rather than amplification of the instruction contrast.

c) Recovery and low-level control: Motion planning, Cartesian servoing [21], and task-specific recovery policies can drive a robot directly toward a desired configuration. Such methods generate or override low-level actions outside the task policy. Recovery Chaining [22] learns a recovery policy that drives the system into a nominal controller’s initiation set [13], [14]; Causeway performs the analogous re-entry by steering the frozen policy’s own representation, adding no policy. It uses the re-entry pose only to define an action-space objective whose gradient steers the representation—the frozen VLA still generates every action—and thus lies between language-only guidance and external control.

III. TASK ISLANDS AND ACCESSIBILITY

A. Problem Formulation

Let $\pi(a \mid x, \ell)$ be a language-conditioned policy over observations x and instructions ℓ , trained by imitation on a set of tasks $\{\ell_k\}$ with demonstration sets $\{\mathcal{D}_k\}$. Running π closed-loop under a fixed ℓ_k from the initial state distribution induces a trajectory distribution $p_k^\pi(\tau)$; task k succeeds when its goal predicate $G_k(\tau)$ holds at some step, and we use the symbol $\rho_k = \Pr_\pi[G_k(\tau) = 1]$ for the policy’s success rate on k from the initial state.

In *instruction switching*: the policy executes ℓ_A from the initial state, and at some state s along that execution the instruction switches permanently to ℓ_B . For such a state, let

$$\rho_{A \rightarrow B}(s) = \Pr_\pi[G_B(\tau) = 1 \mid s, \ell_B] \quad (1)$$

be the probability that the goal of B is subsequently reached, with the from-scratch success ρ_B as the reference. A *task island* refers to a state s at which $\rho_{A \rightarrow B}(s)$ is low, although ρ_B is high: the policy is competent at B , yet cannot begin

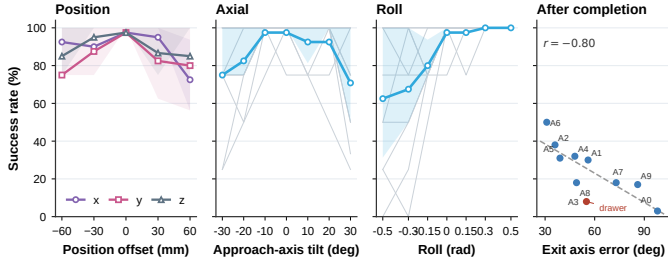


Fig. 2: Entry-basin tolerance and terminal-state mismatch. Left three panels: success under component-wise perturbations around a re-entry pose; thin lines show individual tasks and bands show interquartile ranges. Right: bare-switch success after A completes vs. the source’s exit approach-axis error ($r = -0.80$); A3 (red): low success despite moderate axis error—its hand exits inside the drawer.

it from s . Thus, a task island is defined relative to a target task, policy, and rollout horizon. The remainder of this section makes this outcome-level definition usable, and explains the failure it names. Sec. III-B introduces an object-centric lens—the state ξ , the entry basin, and its measured tolerances—and uses it to construct a computable geometric proxy for Eq. (1). Secs. III-C and III-D locate islands behaviorally—during execution and after task completion—and tie their occurrence to the geometry of ξ . Sec. III-E then asks *why* a new instruction alone does not escape an island, and *what* remains steerable; Sec. IV turns these answers into Causeway.

B. Object-Centric State and Entry Basin

a) *Object-centric state:* Task B manipulates an object o_B and, for pick-and-place tasks, moves it to a destination d_B . We represent the state relevant to B as

$$\xi^B = (p_{EE} - p_{r_B}, R_{EE}, g), \quad r_B = \begin{cases} o_B & \text{before contact,} \\ d_B & \text{after the grasp,} \end{cases} \quad (2)$$

where p_{EE} and R_{EE} are the world-frame end-effector position and orientation; p_{r_B} is the position of task reference r_B , and g is the gripper state. Changing the reference from o_B to d_B keeps ξ^B task-relative during approach and transport. Orientation remains in the world frame because object orientations are fixed in our evaluation settings. Consequently, the same physical state maps to a different ξ^B for each target task.

b) *Demonstrated state set and entry basin:* Let $\mathcal{D}_B$ be the demonstrations of task B and $\mathcal{M}_B = \{\xi^B(s) : s \in \mathcal{D}_B\}$ the *demonstrated state set* in ξ -space. The *entry basin* $\mathcal{S}_B(g)$ is the phase-compatible subset selected by the gripper state g : the approach segment up to first contact when the gripper is open, and the beginning of the placement segment when it already holds o_B . We expand these demonstrated states component-wise using the measured position, approach-axis, and roll tolerances in Fig. 2; each component lies within tolerance of a demonstrated state in the corresponding phase. Thus, $\mathcal{S}_B(g)$ is a demonstration-derived region of states compatible with entering the corresponding phase of task B . Direct

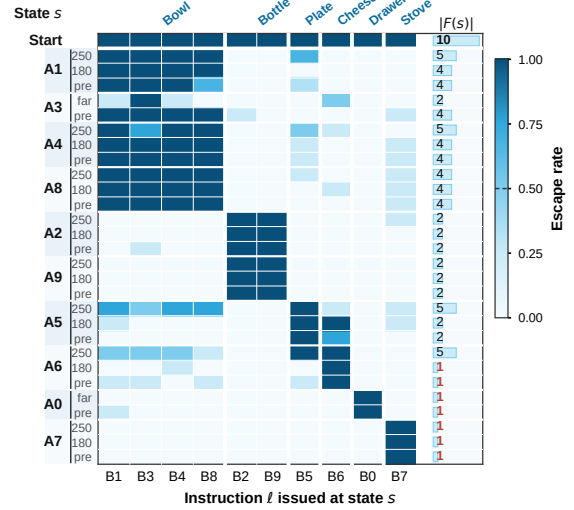


Fig. 3: Escape map (OpenVLA-OFT, LIBERO-Goal). 250, 180: hand-object distance, mm; pre: pre-grasp; far/pre for the two drawer tasks. Columns: the instruction issued at that state, grouped by target object. Cells: fraction of trials reaching the object of ℓ . Right: $|F(s)|$, the number of instructions with escape rate ≥ 0.5 ; $|F(s)|=1$ (red) marks committed states.

placement at a demonstrated pre-grasp configuration recovers task completion (Placement, Table I).

c) *Executable set and task islands:* We operationalize the *empirical executable set* of a state s as the instructions whose demonstrated state set contains it:

$$F(s) = \{\ell : \xi^\ell(s) \in \mathcal{M}_\ell\}. \quad (3)$$

Because g is a component of ξ , membership in $\mathcal{M}_\ell$ already enforces gripper-phase compatibility. For example, an open-gripper state cannot lie in a closed-gripper segment, so Eq. (3) requires no separate phase conditions. We treat a switch from A to B at state s as viable when $\ell_B \in F(s)$. Empirically, $\rho_{A \rightarrow B}(s)$ is low when $\xi^B(s) \notin \mathcal{M}_B$, which gives our geometric operationalization of a task island. As the policy executes A , $\xi^\ell(s_t)$ leaves the demonstrated state sets of other tasks and $|F(s_t)|$ generally contracts.

d) *Validation of the entry basin:* The nominal rate ρ_B from Sec. III-A serves as the *reference level*: any return is evaluated against this level, not against 100% success. When the end effector is placed directly at a pose in $\mathcal{S}_B$ and ℓ_B is issued, the policy—which conditions only on the current observation—recovers this level while the scene remains near B ’s demonstrated configuration; completion of A can alter the scene (Table I). The same holds for poses sampled at different points along the approach segment, showing that $\mathcal{S}_B$ has non-zero extent and the return target is not a specific pose.

e) *Tolerance bands of the basin:* Starting from a pose placed directly on the demonstrated approach (e.g., 120 mm) before contact, we perturb one component of ξ at a time and rerun the policy (Fig. 2; 10 tasks $\times$ 4 trials per point). Success remains near the reference level for position offsets within

30 mm, while the approach axis tolerates about $\pm 20^\circ$. Roll is asymmetric: $+0.5\text{rad}$ shows no measurable loss, whereas -0.15rad already costs 20pp. Gripper state is discrete: a pre-grasp pose with a closed gripper lies outside $\mathcal{S}_B$. These component-wise differences motivate use of separate write channels and handoff criteria of Sec. IV.

C. Islands under Mid-Execution Switching

We next ask when ℓ_B remains executable as the policy performs A .

a) *Measurement: the escape map*: We probe executability at a state s by rolling the policy out closed-loop for a fixed horizon under each instruction ℓ and recording whether the end effector reaches its target object. The resulting state-by-instruction table is the *escape map*; thresholding its escape rates provides a behavioral estimate of the geometrically defined $F(s)$ (Fig. 3). We sample states from the initial pose through approach, pre-grasp, and grasp, and construct the detailed map on OpenVLA-OFT because its instruction response attenuates most strongly along the approach (Fig. 4a); cross-model generality is evaluated in Sec. V. In all tested cases, short-horizon reachability agrees with full-length rollouts.

b) *Object identity structures surviving tasks*: At the shared initial pose all ten instructions are executable; as the policy executes A , most cross-object instructions fall out of $F(s)$ (Fig. 3). The remaining set is strongly structured by object identity: at the final sampled state, all four bowl-source tasks retain the four bowl instructions, while both bottle-source tasks retain the two bottle instructions. Several singleton-object sources reach $|F(s)|=1$, where only one instruction remains. The evolution need not be monotone: along A3, $|F(s)|$ falls from 10 to 2 and then rises to 4 as the remaining bowl instructions cross the escape threshold again. This rebound shows that executability follows the state rather than elapsed time. The same-object case is followed beyond pre-grasp below.

c) *Same-object pairs*: When A and B manipulate the same object ($o_A=o_B$), post-grasp viability depends on whether their demonstrated grasp and early-transport states overlap. A switch can survive the grasp while those states remain compatible, then fail as the transport paths diverge. Without such overlap, the held state may become an island immediately.

D. Islands after Task Completion

The terminal state induced by A can itself be an island for B . Across source tasks, bare-switch success after completion ranges from 3% to 50% and is strongly negatively associated with the mean target-relative approach-axis error at exit, which spans 31° – 98° ($r=-0.80$; Fig. 2). A3 is an outlier because the hand ends inside the drawer region despite a mid-range axis error; A7 also ends with the gripper closed. Terminal mismatch can therefore involve several components of ξ^B , motivating the orientation channels and release step in Sec. IV.

E. The Representation Gate

The preceding analysis locates task islands. We now examine why a new instruction alone may not escape them.

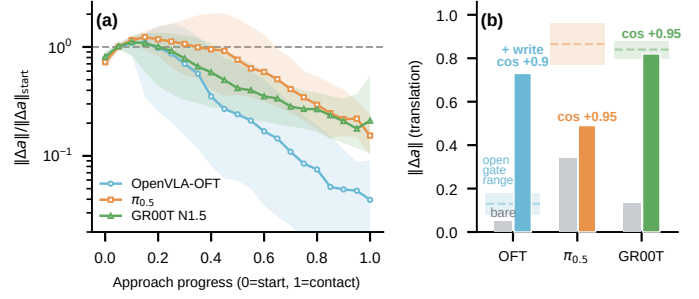


Fig. 4: **Instruction influence attenuates along the approach, yet the gated representation remains steerable.** (a) Action response to swapping $\ell_A \rightarrow \ell_B$, relative to the start of the approach (median, IQR). (b) At post-approach states, a gradient direction computed on the current forward pass restores the response toward the open-gate range and aligns the decoded motion with the return direction.

a) *State attenuates the instruction’s effect on action*: At a fixed observation along A ’s trajectory, we run the policy with ℓ_A and ℓ_B and measure the action response $\|\Delta a\|$ as the norm of the mean translational difference between the two decoded action chunks. The new instruction always changes the text-token representation, but its effect on action depends on state. Before contact, $\|\Delta a\|$ drops by one to two orders of magnitude on all three architectures (Fig. 4a)—beyond what converging near-optimal actions explain, since a gradient direction at the same states still produces motion toward B (Fig. 4b); we call this state-conditioned attenuation the *gate*. It appears along the same approach over which $F(s)$ changes (Sec. III-C), linking the behavioral and representational findings.

b) *The action representation remains steerable at gated states*: When the instruction no longer changes the action, a gradient from the current forward pass restores the action response and aligns the decoded motion with the return direction on all three architectures (Fig. 4b). This result identifies both the recovery target and intervention point: steer the action stream until $\xi^B(s)$ returns to $\mathcal{S}_B(g)$ (Sec. IV).

IV. BRIDGING TASK ISLANDS BY REPRESENTATION STEERING

Section III suggests three design choices: return to $\mathcal{S}_B(g)$, intervene in the action-stream representation, and use a state-directed gradient when the instruction’s influence is attenuated. Causeway realizes all three: at test time it consumes only the robot’s proprioceptive state and a re-entry pose T^* precomputed from the demonstrations of B , and every action is decoded by the frozen policy. A *representation write* moves the decoded action toward a target pose in $\mathcal{S}_B(g)$ (Sec. IV-A), while a proprioceptive rule controls *engagement and handoff* (Sec. IV-B). Separate channels handle position, approach axis, roll, and gripper state because each has a different re-entry tolerance.

A. Representation Write

a) *Write location*: For each architecture, we write within the action stream identified in Sec. III-E: one layer L on $\pi_{0.5}$ and GR00T N1.5, and multiple layers $\mathcal{L}$ on OpenVLA-OFT. Below, L denotes a generic write location; on OpenVLA-OFT the same operation is applied at each $L \in \mathcal{L}$. Let $h_L^{(k)} \in \mathbb{R}^{N \times D}$ denote the hidden state of the N action tokens at layer L during decoding step k , where D is the hidden dimension and $k = 1, \dots, K$. The final decoded action chunk is

$$a = f_L(h_L^{(1)}, \dots, h_L^{(K)}), \quad a \in \mathbb{R}^{H \times 7}, \quad (4)$$

where f_L denotes the remaining decoding computation from L to the final action, H is the action-chunk horizon, and each decoded action comprises translation and rotation increments ($\mathbb{R}^3$ each) and one gripper command. For iterative action heads, $K > 1$; for a parallel-decoding head, $K = 1$ and Eq. (4) reduces to a single forward pass.

b) *Target*: $T^* = (p^*, R^*)$ is chosen from the phase-compatible segment of $\mathcal{S}_B(g)$ by the same fixed rule for all task pairs and models, without pair-specific tuning. A local neighborhood of T^* defines the handoff condition (Sec. IV-B).

c) *Return geometry*: Let (p, R) denote the current end-effector pose, corresponding to (p_{EE}, R_{EE}) in Eq. (2). The return geometry toward T^* is computed by the translational and rotational offsets

$$e^p = p^* - p, \quad e^r = \log(R^* R^{-1})^\vee, \quad (5)$$

where $(\cdot)^\vee$ maps a skew-symmetric matrix in $\mathfrak{so}(3)$ to its corresponding three-dimensional rotation vector. Both errors are expressed in the world frame. We compute the rotational error from the relative rotation $R^* R^{-1}$, consistent with the policy's left-multiplicative rotation increment.

d) *Gradient write*: Let $\delta \in \mathbb{R}^D$ denote a shared representation perturbation. At each decoding step k , the same perturbation is broadcast to all N action tokens at layer L ,

$$\tilde{h}_L^{(k)} = h_L^{(k)} + \mathbf{1}_N \delta^\top, \quad k = 1, \dots, K, \quad (6)$$

where $\mathbf{1}_N \in \mathbb{R}^N$ is the all-ones vector. The same δ is shared across all decoding steps and, on OpenVLA-OFT, across every write layer $L \in \mathcal{L}$. Let $a(\delta)$ denote the final action chunk produced by the complete decoding process under all such injections. Each channel c defines a scalar objective $J^c(a)$ as the component of the predicted motion along its desired direction (e.g., u^p for position, defined below). The corresponding representation-space direction is

$$\hat{g}^c = \frac{\nabla_\delta J^c(a(\delta))}{\|\nabla_\delta J^c(a(\delta))\|} \Big|_{\delta=0}. \quad (7)$$

Each channel is normalized separately, so relative strengths are set only by the per-channel gains introduced below. The gradient is computed through the complete decoding computation. On OpenVLA-OFT's parallel-decoding head, this reduces to a single forward-backward pass.

1) *Channel Decomposition*: Let the h -th decoded action be $a_h = (a_{h,xyz}, a_{h,rot}, a_{h,g})$, for $h = 1, \dots, H$. We aggregate each component over the action chunk as

$$\bar{a}_{xyz} = \sum_{h=1}^H a_{h,xyz}, \quad \bar{a}_{rot} = \sum_{h=1}^H a_{h,rot}, \quad \bar{a}_g = \sum_{h=1}^H a_{h,g}. \quad (8)$$

Let $\hat{z}_w = R\hat{z}$ denote the current gripper approach axis in the world frame, where $\hat{z}$ is the unit approach axis in the gripper frame. We define separate objectives for translation, approach-axis alignment, roll, and gripper motion.

a) *Position channel*: Let $u^p = e^p / \|e^p\|$ denote the unit direction toward p^* . We define

$$J^p = \langle \bar{a}_{xyz}, u^p \rangle. \quad (9)$$

Substituting J^p into Eq. (7) gives $\hat{g}^p$, the representation direction that locally increases decoded motion toward p^* . The remaining channels are constructed in the same way from their corresponding objectives.

b) *Orientation decomposition*: We decompose the rotational error e^r into a component that changes the gripper approach direction and a component about that direction:

$$e^\perp = e^r - (e^r \cdot \hat{z}_w) \hat{z}_w, \quad (10)$$

$$r = e^r \cdot \hat{z}_w. \quad (11)$$

Here, $e^\perp$ changes the approach direction and r is the signed roll error about the current approach axis.

c) *Approach-axis channel*: Let $u^\perp = e^\perp / \|e^\perp\|$. We define

$$J^\perp = \langle \bar{a}_{rot}, u^\perp \rangle, \quad (12)$$

which yields the representation direction $\hat{g}^\perp$.

d) *Roll channel*: The desired roll direction is $\text{sign}(r)\hat{z}_w$, giving

$$J^r = \langle \bar{a}_{rot}, \text{sign}(r)\hat{z}_w \rangle. \quad (13)$$

The resulting representation direction is $\hat{g}^r$. Because successful re-entry is sensitive to roll error (Sec. III), we taper its gain near zero:

$$\alpha_r(r) = \alpha_r^{\max} \min\left(1, \frac{|r|}{\kappa}\right) \mathbb{1}_{[|r| > \epsilon]}, \quad (14)$$

where κ defines the proportional band and ϵ the dead-band threshold.

e) *Gripper channel*: When the hand holds o_A and $o_A \neq o_B$, the return begins by opening the gripper: $J^g = \bar{a}_g$ on the gripper command channel, which yields $\hat{g}^g$. If the hand already holds o_B , the closed-gripper branch of $\mathcal{S}_B(g)$ is used instead.

f) *Composition*: The write applied at inference step t is

$$\delta_t = \begin{cases} \alpha_g \hat{g}^g, & \text{during release,} \\ \alpha_p \hat{g}^p + \alpha_\perp \hat{g}^\perp + \alpha_r(r) \hat{g}^r, & \text{otherwise.} \end{cases} \quad (15)$$

At each inference step, the channel directions are evaluated at $\delta = 0$, combined according to Eq. (15). The final action is decoded by the policy from the shifted representation (gain values in Sec. V-A).

B. Engagement and Handoff

The representation write is scheduled from proprioception and the target pose $T^* = (p^*, R^*)$. Let $\hat{z}_w^* = R^* \hat{z}$ and we define

$$T_0 = (p_0, R^*), \quad p_0 = p^* - d_0 \hat{z}_w^*. \quad (16)$$

a) Engagement and return: The write starts at the instruction switch. If the hand holds $o_A \neq o_B$, only the gripper channel is active until the hand opens beyond $\bar{g}$; otherwise, the position and orientation channels engage immediately. They first drive the end effector toward T_0 while aligning with R^* . Once p_0 is reached and the approach axis is aligned, the position target switches to p^* when $|r| < \gamma$, or after N_γ additional steps. Upon entering the neighborhood of T^* , the write continues for at most N_d steps; the roll channel is disabled during this dwell to avoid constraining the policy’s subsequent wrist motion.

b) Handoff: The write disengages after the dwell or earlier when the gripper remains closed around a non-empty grasp for two consecutive steps inside the neighborhood of T^* . A closure outside this neighborhood does not trigger handoff; the gripper channel reopens the hand and the return continues.

This schedule determines only *when* the write is active and *which* channels engage; it issues no motor commands; every action, during re-entry and after handoff, is decoded by the frozen policy.

V. SIMULATION EXPERIMENTS

We test four hypotheses: task islands occur across models and switch timings (H1); Causeway restores access through re-entry (H2); neither instruction contrast nor action-level steering is sufficient (H3); the state-compatibility account extends beyond the main cross-object setting (H4).

A. Setup

a) Benchmarks: We evaluate LIBERO-Goal’s 90 ordered task pairs: the main evaluation uses the 71 cross-object pairs, and the 14 same-object pairs serve H4. Five pairs are excluded because executing A displaces B ’s target configuration. We also evaluate LIBERO-Object to test generalization beyond LIBERO-Goal.

b) Policies: We evaluate benchmark-fine-tuned checkpoints of OpenVLA-OFT [23] (parallel decoding), $\pi_{0.5}$ [24] (flow matching), and GR00T N1.5 [5] (diffusion). Writes target the action-stream layers diagnosed in Sec. III-E: layers $\{8, 12, 16, 20, 24, 28, 30\}$ for OpenVLA-OFT, layer 15 for $\pi_{0.5}$, and DiT block 16 for GR00T. OpenVLA-OFT and GR00T use a single gain profile across all pairs. For $\pi_{0.5}$, three fixed geometry-driven profiles are used: grasp-and-place targets use a larger translation gain, while sources A0 and A9 use a stronger orientation gain to counter post-completion wrist-angle errors (Sec. III-D). All gain profiles remain fixed across pairs and switch timings, and the schedule thresholds ($\gamma = 0.3$ rad, $N_\gamma = 20$, $N_d = 15$ steps; Sec. IV-B) are additionally shared across all three models.

c) Protocol and metric: Policies execute ℓ_A from benchmark initial states before switching to ℓ_B at step 30 (approach), step 60 (object held in 58–61% of trials), or post-completion. Evaluating 71 pairs across 3 switch conditions, 8 initial states, and 3 rollouts yields $71 \times 3 \times 8 \times 3 = 5,112$ trials per model. Because switch states vary, success is evaluated over the state distribution p_A^π induced by task A at switch timing $\tau \in \{30, 60, \text{done}\}$:

$$\rho_{A \rightarrow B}^{(\tau)} = \mathbb{E}_{s_\tau \sim p_A^\pi} [\rho_{A \rightarrow B}(s_\tau)],$$

under standard LIBERO criteria. All models achieve near-perfect from-scratch success on target tasks from clean initial states.

d) Inference cost: On one A100-80GB, per-chunk latency rises $2.5\text{--}2.8 \times$ ($102/75/128 \rightarrow 253/187/362$ ms; replanning $4.0/5.3/2.8$ Hz; ± 4.3 GiB peak) on OFT/ $\pi_{0.5}$ /GR00T, active only during re-entry.

e) Baselines: We compare against four steering baselines: (1) *Bare switch*, which directly replaces ℓ_A with ℓ_B ; (2) *Contrastive prompt guidance (CPG)* [17], which combines predictions via $v_A + w(v_B - v_A)$; (3) *Contrastive activation addition (CAA)* [19], which injects online differences $d_L \propto \bar{h}_L(x_t, \ell_B) - \bar{h}_L(x_t, \ell_A)$ at each step through Causeway’s write machinery; and (4) *Action-space steering*, which applies the state-directed signal and schedule as a Cartesian residual directly on the decoded actions, evaluated on $\pi_{0.5}$, where the baseline signal is strongest and floor effects are smallest.

f) References with Privileged recovery control: We also report references with privileged recovery control on OpenVLA-OFT. *State placement* sets the robot to a demonstration pre-grasp configuration of B at the switch while preserving the scene. The *single-point* and *wrist-first* servos drive the arm to the same configuration with an external controller, the latter along a hand-designed wrist-first path. These are references rather than baselines: they rely on state manipulation, external control, or hand-designed motion that Causeway does not use.

B. Results

a) H1: Causeway bridges task islands: Across 71 cross-object pairs and three switch timings, bare-switch success averages 26.0% for $\pi_{0.5}$, 3.4% for OpenVLA-OFT, and 9.9% for GR00T (Table I), despite near-perfect from-scratch performance. Causeway raises success to 61.6% (+35.6 percentage points), 64.5% (+61.1), and 47.5% (+37.6), respectively. Gains are positive for every model, timing, and target (Fig. 5), ranging from +28.6 points for GR00T after completion to +72.1 for OpenVLA-OFT at step 30.

b) H2: Causeway restores access through re-entry: Causeway raises the rate of reaching the handoff neighborhood by 42–72 percentage points and increases completion conditioned on reaching it for all three models (Table I, Fin|R). Thus, it both reaches the neighborhood more often and arrives in states from which the frozen policy completes B more reliably. T^* is a representative target within the entry basin, not a uniquely tuned recovery pose (Section III-B).

TABLE I: **Switch success (%)**. ℓ_B issued at Step 30, Step 60, or after A completes. *Reach*: fraction reaching the handoff neighborhood of T^* ; *Fin|R*: completion given reach. Gray: references using external state manipulation. Best steering method per block in bold.

Model	Method	Switch point				Re-entry to T^*	
		Step 30	Step 60	After	Avg.	Reach	Fin R.
$\pi_{0.5}$	Bare	29.9	22.9	25.1	26.0	18.7	58.7
	CPG	35.2	21.1	26.5	27.6	—	—
	CAA	26.9	21.7	26.4	25.0	—	—
	Action	37.5	21.3	26.6	28.5	—	—
	Ours	60.9	60.4	63.4	61.6	61.0	74.0
OpenVLA-OFT	Bare	5.1	2.9	2.2	3.4	9.3	25.3
	CPG	10.3	6.7	4.0	7.0	—	—
	CAA	0.0	0.0	2.5	0.8	—	—
	Ours	77.2	60.5	55.9	64.5	81.3	73.7
GR00T N1.5	Bare	14.6	11.8	3.4	9.9	8.7	45.0
	CPG	13.2	10.0	7.7	10.3	—	—
	CAA	5.0	5.0	3.8	4.6	—	—
	Ours	67.2	43.3	32.0	47.5	79.0	59.3
References using external state manipulation	<i>Servo (single-point)</i>	67.5	49.2	25.0	47.2	—	—
	<i>Servo (wrist-first)</i>	97.0	76.0	48.0	73.7	—	—
	<i>Placement</i>	95.0	93.1	49.4	79.2	—	—

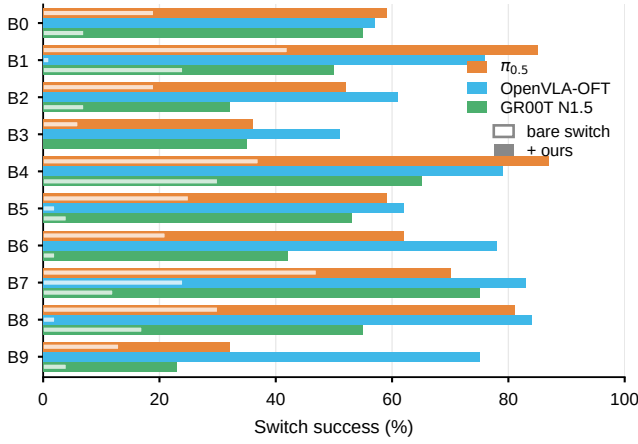


Fig. 5: Per-target switch success, pooled over source tasks, switch timings, initial states, and rollouts. Filled: with the write; white inset: bare switch. Bottom: pooled average.

c) *H3: neither instruction contrast nor action-level steering overcomes the gate*: Prompt-level contrast provides no consistent recovery: CPG shifts average success by at most +3.6 pp and reduces it at some timings (Table I). Representation-level contrast fails more decisively. CAA uses the same write machinery as Causeway and differs only in direction, yet performs at or below the bare switch on all three models on average (Table I). Consistently, at gated states the gradient write induces target-aligned motion while the instruction-difference direction does not (Fig. 4b).

The same state-directed signal also performs poorly when applied only at the action level. On $\pi_{0.5}$, a Cartesian residual improves the step-30 switch but falls near bare at step 60 and after completion, while the representation write exceeds 60% at all three timings. Each alternative therefore lacks at least one of two ingredients: state-directed steering or representation-level injection.

TABLE II: **Real-robot instruction switching** (xArm, $\pi_{0.5}$ fine-tuned). Success rate (%) over 5 trials per cell. ℓ_B is issued during the approach, at the pre-grasp pose, or after A completes. Tasks: press *button*, put the bowl on the *plate / shelf*, put the corn in the drawer (*corn*). †: same-object pairs (both move the bowl). *Solo*: the policy’s own success on B from the standard start.

$A \rightarrow B$	Approach		Pre-grasp		After A	
	bare	ours	bare	ours	bare	ours
button $\rightarrow$ shelf	100	100	0	100	40	80
shelf $\rightarrow$ button	0	100	0	80	0	60
button $\rightarrow$ plate	0	100	0	100	0	100
plate $\rightarrow$ button	0	100	0	80	0	100
corn $\rightarrow$ plate	0	100	0	100	80	100
plate $\rightarrow$ shelf†	100	100	0	100	0	60
shelf $\rightarrow$ plate†	100	100	100	100	0	80
Pooled	42.9	100	14.3	94.3	17.1	82.9

Solo (%): button 80, plate 100, shelf 100, corn 80.

d) *Privileged recovery references*: On OpenVLA-OFT, Causeway (64.5%) outperforms even the externally controlled single-point servo (47.2%). Stronger privileged references perform better: the hand-designed wrist-first servo reaches 73.7%, and direct state placement 79.2% (Table I). Thus, matching or exceeding Causeway requires additional external control, hand-designed motion, or direct state manipulation.

e) *H4: state compatibility beyond the main setting*: Same-object switches follow the state-compatibility account of Sec. III-C: switches remain viable while the two tasks share compatible holding states and fail after their trajectories diverge. On LIBERO-Object, the write lifts the committed-state switch from 0% to 57.5% on the evaluated pairs, extending the recovery beyond the main cross-object setting.

VI. REAL-WORLD EXPERIMENTS

We validate Causeway on an xArm 6 with $\pi_{0.5}$, fine-tuned on 30 teleoperated demonstrations per task and frozen at inference; re-entry poses T^* are extracted by the rule of Sec. IV-A. We evaluate four tasks—**button**, **plate**, **shelf**, and **corn** (80–100% solo success; Table II)—across 7 ordered pairs (5 cross-object, 2 same-object). Target instructions ℓ_B are issued at three transition points: mid-approach, at pre-grasp, or post-completion (5 trials/cell).

a) *Islands persist and Causeway bridges them*: Bare-switch success collapses as the robot approaches the object—from 42.9% mid-approach to 14.3% and 17.1% at pre-grasp and post-completion (Table II)—due to policy stalling or continuing toward the source object (Fig. 6). Causeway elevates pooled success to 82–100%; every cell reaches at least 60%, and zero-success pairs recover to 80–100%.

b) *Held objects and chained switching*: For same-object pairs with an already-held item, Causeway uses the closed-gripper branch of $\mathcal{S}_B(g)$ (Sec. IV-B) to route directly to the target placement segment—e.g., re-routing a held bowl from plate to shelf without put-down or re-grasp (Fig. 6a).

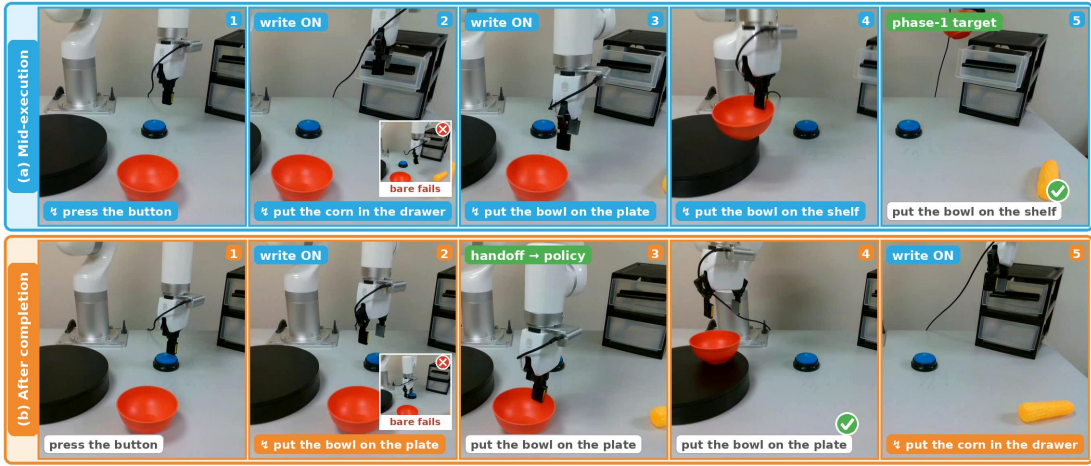


Fig. 6: **Real-robot chained switching** (xArm, $\pi_{0.5}$, fine-tuned then frozen): *Top Row*: Each instruction interrupts the previous task mid-execution; the final switch re-routes the held bowl from the plate to the shelf via the phase-conditioned target (closed-gripper branch, Sec. IV-B); *Bottom Row*: Instructions issued after each task completes. Solid instruction chips mark the frame where a new instruction takes effect; insets show the bare switch failing from the same state. Quantitative pairwise results in Table II.

Across chained task sequences, Causeway enables continuous language-driven redirection mid-execution or post-completion without resets, whereas bare switches fail.

VII. CONCLUSION, LIMITATIONS, AND FUTURE WORK

We characterize task islands—states from which a competent policy cannot begin a new task—and introduce Causeway, a training-free, inference-time method that applies a representation write by back-propagating through a frozen policy’s decoding during return motion. Across three VLA architectures on LIBERO-Goal, Causeway elevates bare-switch success from 3–26% to 47–65%, and transfers to LIBERO-Object and a physical xArm. Causeway depends on task-relative coordinates (ξ) and demonstration coverage, with performance degrading under unmodeled factors (such as large scene shifts or in-grasp dynamics) and requiring per-architecture calibration. Future work will use these diagnostics to embed capability accessibility directly into policy training.

REFERENCES

- [1] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 40 417–40 439.
- [2] A. Brohan, N. Brown, J. Carbajal *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv:2307.15818*, 2023.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv:2406.09246*, 2024.
- [4] K. Black, N. Brown, D. Driess *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv:2410.24164*, 2024.
- [5] NVIDIA, J. Bjorck *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv:2503.14734*, 2025.
- [6] Z. Chen, A. Tian, L. Wang, B. Joffe, Y. C. Lin, Y. Chen, S. Karamcheti, and D. Xu, “Resteer: Quantifying and refining the steerability of multitask robot policies,” 2026.
- [7] M. Li, Z. Zhao, Z. Che, F. Liao, K. Wu, Z. Xu, P. Ren, Z. Jin, N. Liu, and J. Tang, “Switchvla: Execution-aware task switching for vision-language-action models,” 2025.
- [8] Octo Model Team, D. Ghosh, H. Walke *et al.*, “Octo: An open-source generalist robot policy,” *arXiv:2405.12213*, 2024.
- [9] W. Liufu, X. Guo, R. Chen *et al.*, “Repo-vla: Recovery-driven policy optimization for vision-language-action models,” *arXiv:2605.09410*, 2026.
- [10] D. M. S. Tan, S. Shailesh, S. Iyer, W. W. J. Teo, Y. Ju, Q. Gu, and G. Sartoretti, “Rl²-vla: Adaptive rl latent compositional steering with test-time scaling for vision-language-action models,” 2026.
- [11] M. Ahn, A. Brohan, N. Brown *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, 2022.
- [12] L. X. Shi, Z. Hu, T. Z. Zhao *et al.*, “Yell at your robot: Improving on-the-fly from language corrections,” in *Robotics: Science and Systems*, 2024.
- [13] R. S. Sutton, D. Precup, and S. Singh, “Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning,” *Artificial Intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999.
- [14] G. Konidaris and A. Barto, “Skill discovery in continuous reinforcement learning domains using skill chaining,” in *Advances in Neural Information Processing Systems*, 2009.
- [15] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv:2207.12598*, 2022.
- [16] X. L. Li, A. Holtzman, D. Fried *et al.*, “Contrastive decoding: Open-ended text generation as optimization,” in *Association for Computational Linguistics*, 2023.
- [17] S. Huang, J. Shao, K. Wang *et al.*, “Breaking lock-in: Preserving steerability under low-data vla post-training,” *arXiv:2604.23121*, 2026.
- [18] A. M. Turner, L. Thiergart, D. Udell *et al.*, “Activation addition: Steering language models without optimization,” *arXiv:2308.10248*, 2023.
- [19] N. Rimsky, N. Gabrieli, J. Schulz *et al.*, “Steering llama 2 via contrastive activation addition,” in *Association for Computational Linguistics (ACL)*, 2024.
- [20] A. Zou, L. Phan, S. Chen *et al.*, “Representation engineering: A top-down approach to ai transparency,” *arXiv:2310.01405*, 2023.
- [21] F. Chaumette and S. Hutchinson, “Visual servo control, part i: Basic approaches,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006.
- [22] S. Vats, D. K. Jha, M. Likhachev, O. Kroemer, and D. Romeres, “Recoverychaining: Learning local recovery policies for robust manipulation,” *arXiv:2410.13979*, 2025.
- [23] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv:2502.19645*, 2025.
- [24] Physical Intelligence, K. Black *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv:2504.16054*, 2025.